Los 10 cruces con más fatalidades a peatones — ATUS ZMM

Qué cruces de la Zona Metropolitana de Monterrey concentran más hechos de tránsito en los que un peatón o un ciclista fue arrollado o murió, 2019-2024. Una ficha por cruce: su nombre, cuántos hechos de tránsito tiene, qué parte de ellos le pegó a alguien a pie o en bicicleta, cómo se reparten por trimestre, la foto satelital y el lugar que ocupa entre todos los cruces.

Fuente: `data/processed/atus_zmm_limpio.parquet` (se genera con `uv run limpiar-atus`). El mismo notebook escribe la página `docs/cruces_peatones/index.html`, que muestra las mismas fichas con el estilo de la presentación.

Qué hay aquí

§	Tema
1	La unión: qué cuenta como hecho de peatón o ciclista
2	La tabla de cruces
3	El ranking: los 10 cruces
4	Una ficha por cruce
5	La página HTML

> **La unidad es el cruce, como en `patrones_espaciales.qmd`.** Un cruce es un nodo `id_punto_4d` (10 m de tolerancia) cuya ubicación más frecuente es «Intersección». Su nombre es el par de vialidades que más veces aparece en sus partes.

```
import base64
import io
import json
import re
import time

import contextily as cx
import numpy as np
import pandas as pd
import requests
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec
from IPython.display import Image as ImagenIPy, display
from PIL import Image
from pyproj import Transformer
```

```
from geostats import rutas

d = pd.read_parquet(rutas.ATUS_ZMM_LIMPIO)
N = len(d)
print(f"{N:,} hechos de tránsito | {d.fecha.min():%Y-%m-%d} a {d.fecha.max():%Y-%m-%d} | "
      f"{d.CVE_MUN.nunique()} municipios")
```

```
379,294 hechos de tránsito | 2019-01-01 a 2024-12-31 | 18 municipios
```

Estilo de las gráficas

Misma identidad que el resto de los notebooks y que la presentación.

```
# --- Identidad GeoStats -------------------------------------------------------
AZUL     = "#005991"   # Azul Prusia ajustado -> datos
ROJO     = "#8B2C1A"   # Rojo profundo -> énfasis, títulos
OXIDO    = "#B15E2E"   # Rojo óxido -> detalle cálido
GRAFITO  = "#2C2C2C"   # texto y estructura
GRIS     = "#F2F2F2"   # fondos y rejilla
BLANCO   = "#FFFFFF"
FONDO_PTS = "#DCDCDC"

TITULAR = ["Montserrat", "Helvetica Neue", "Arial", "DejaVu Sans"]
TEXTO   = ["Cormorant Garamond", "EB Garamond", "Georgia", "DejaVu Serif"]
CIFRAS  = ["Roboto Mono", "Menlo", "DejaVu Sans Mono"]

plt.rcParams.update({
    "figure.dpi": 130, "figure.facecolor": BLANCO, "axes.facecolor": BLANCO,
    "savefig.facecolor": BLANCO, "font.family": "sans-serif",
    "font.sans-serif": TITULAR, "font.serif": TEXTO, "font.monospace": CIFRAS,
    "font.size": 9, "axes.edgecolor": GRIS, "axes.labelcolor": GRAFITO,
    "axes.labelsize": 9, "axes.titlecolor": ROJO, "axes.titlesize": 11.5,
    "axes.titleweight": "bold", "axes.titlelocation": "left", "axes.titlepad": 12,
    "axes.grid": True, "axes.axisbelow": True, "grid.color": GRIS,
    "grid.linewidth": 1.0, "xtick.color": GRAFITO, "ytick.color": GRAFITO,
    "xtick.labelsize": 8, "ytick.labelsize": 8, "legend.frameon": False,
    "legend.fontsize": 8, "lines.linewidth": 1.8,
})


def pulir(ax, ejes=("top", "right", "left")):
    """Quita marco y pone las cifras en monoespaciada."""
    for lado in ejes:
        ax.spines[lado].set_visible(False)
    ax.tick_params(length=0)
    for etiqueta in ax.get_xticklabels() + ax.get_yticklabels():
        etiqueta.set_fontfamily("monospace")
    return ax
```

```
def miles(n) -> str:
    """Coma de miles, como en la presentación."""
    return f"{int(n):,}"


def pct(x) -> str:
    """Punto decimal y espacio antes del signo, como en la presentación."""
    return f"{x:.1f} %"
```

---

1. La unión: qué cuenta como hecho de peatón o ciclista

El ranking se define con **una sola cuenta por cruce**: cuántos de sus hechos de tránsito caen en la unión de dos conjuntos.

- **Hecho con peatón**: un atropellamiento (`TIPACCID == 2`) **o** cualquier hecho en el que murió un peatón (`PEATMUERTO > 0`).
- **Hecho con ciclista**: una colisión con ciclista (`TIPACCID == 11`) **o** cualquier hecho en el que murió un ciclista (`CICLMUERTO > 0`).

La unión se cuenta **por hecho, no por condición**. Un atropellamiento con un peatón fallecido cumple las dos condiciones del primer conjunto y cuenta una vez; un hecho con un peatón y un ciclista también cuenta una vez. Si se sumaran las condiciones, un cruce con pocos atropellamientos fatales subiría por contar dos veces cada uno.

La segunda condición de cada conjunto no es redundante: hay defunciones de peatón o ciclista registradas con otro tipo de hecho (una colisión entre vehículos que termina sobre la banqueta, por ejemplo). Sin ella esas muertes quedarían fuera de un ranking de fatalidades.

```
d["con_peaton"] = d.TIPACCID.eq(2) | d.PEATMUERTO.gt(0)
d["con_ciclista"] = d.TIPACCID.eq(11) | d.CICLMUERTO.gt(0)
d["union"] = d.con_peaton | d.con_ciclista
d["defunciones_pc"] = d.PEATMUERTO + d.CICLMUERTO
d["heridos_pc"] = d.PEATHERIDO + d.CICLHERIDO

fuera = (d.PEATMUERTO.gt(0) & d.TIPACCID.ne(2)) | (d.CICLMUERTO.gt(0) & d.TIPACCID.ne(11))
print(f"Hechos con peatón:      {miles(d.con_peaton.sum()):>7}")
print(f"Hechos con ciclista:    {miles(d.con_ciclista.sum()):>7}")
print(f"Con los dos:            {miles((d.con_peaton & d.con_ciclista).sum()):>7}")
print(f"Unión:                  {miles(d.union.sum()):>7} ({pct(100 * d.union.mean())} de la ZMM)")
print(f"  entran solo por la defunción, no por el tipo de hecho: {miles(fuera.sum())}")
print(f"Defunciones de peatón o ciclista (personas): {miles(d.defunciones_pc.sum())}")
```

```
Hechos con peatón:      8,582
Hechos con ciclista:    1,512
```

```
Con los dos:                  0
Unión:                   10,094 (2.7 % de la ZMM)
  entran solo por la defunción, no por el tipo de hecho: 30
Defunciones de peatón o ciclista (personas): 475
```

---

2. La tabla de cruces

Una fila por nodo. Se quedan los nodos que son intersección: los tramos sin cruce, los caminos rurales y las carreteras no son cruces, aunque tengan hechos de tránsito.

El nombre: el INEGI captura el mismo cruce de varias formas («Avenida Cristóbal Colón» y «Cristóbal Colón»; a veces con las calles en orden inverso). Se quita el genérico, se ordena el par y se queda el que más veces aparece.

```
GENERICO = re.compile(r"^(AVENIDA|AV\.?|CALLE|CALZADA|BOULEVARD|BLVD\.?|CARRETERA|"
                      r"PROLONGACI[OÓ]N|PRIVADA|ANDADOR)\s+", re.IGNORECASE)
MINUSCULAS = {"De", "Del", "La", "Las", "Los", "Y", "El"}


def calle_limpia(calle) -> str | None:
    if not isinstance(calle, str) or not calle.strip():
        return None
    nombre = GENERICO.sub("", re.sub(r"\s+", " ", calle.strip()))
    palabras = nombre.title().split()
    return " ".join(p.lower() if p in MINUSCULAS and k else p
                    for k, p in enumerate(palabras))


d["c1"] = d.CALLE1.map(calle_limpia)
d["c2"] = d.CALLE2.map(calle_limpia)


def nombre_cruce(grupo: pd.DataFrame) -> str:
    pares = grupo[["c1", "c2"]].dropna()
    if pares.empty:
        sola = grupo.c1.dropna()
        return sola.mode().iloc[0] if len(sola) else "Sin nombre"
    ordenados = pares.apply(lambda r: " × ".join(sorted([r.c1, r.c2])), axis=1)
    return ordenados.mode().iloc[0]


g = d.groupby("id_punto_4d", sort=False)
nodos = pd.DataFrame({
    "total": g.size(),
    "union": g.union.sum(),
    "peatones": g.con_peaton.sum(),
    "ciclistas": g.con_ciclista.sum(),
    "atropellamientos": g.TIPACCID.agg(lambda s: int((s == 2).sum())),
```

```
    "col_ciclista": g.TIPACCID.agg(lambda s: int((s == 11).sum())),
    "defunciones": g.defunciones_pc.sum(),
    "hechos_fatales": g.defunciones_pc.agg(lambda s: int((s > 0).sum())),
    "heridos": g.heridos_pc.sum(),
    "lon": g.LONGITUD.median(),
    "lat": g.LATITUD.median(),
    "municipio": g.NOM_MUN.agg(lambda s: s.mode().iloc[0]),
    "ubicacion": g.ubicacion.agg(lambda s: s.mode().iloc[0]),
})
cruces = nodos[nodos.ubicacion == "Intersección"].copy()

# Lugar por hechos de tránsito entre TODOS los cruces: 1 es el que más tiene.
# Empates comparten el lugar más alto (método «min»), como en una tabla deportiva.
cruces["lugar_total"] = cruces.total.rank(ascending=False, method="min").astype(int)

print(f"{miles(len(nodos))} nodos -> {miles(len(cruces))} cruces "
      f"({pct(100 * cruces.total.sum() / N)} de los hechos de tránsito)")
print(f"Cruces con al menos un hecho de peatón o ciclista: "
      f"{miles((cruces.union > 0).sum())}")
con_def = cruces.defunciones[cruces.defunciones > 0]
print(f"Cruces con al menos una defunción de peatón o ciclista: {miles(len(con_def))}; "
      f"por número de defunciones: {con_def.value_counts().sort_index().to_dict()}")
```

```
54,742 nodos -> 51,305 cruces (97.4 % de los hechos de tránsito)
Cruces con al menos un hecho de peatón o ciclista: 6,620
Cruces con al menos una defunción de peatón o ciclista: 417; por número de defunciones: {1: 385,
2: 31, 3: 1}
```

Las defunciones solas no alcanzan para ordenar: casi todos los cruces con alguna tienen una sola, y el máximo de la ZMM es tres. Ordenar solo por muertes daría cientos de empates en el primer lugar. Por eso la cuenta que ordena es la unión, y las defunciones entran como desempate.

---

3. El ranking: los 10 cruces

Orden: hechos en la unión, de mayor a menor. Si dos cruces empatan, desempata primero quien tiene más defunciones de peatón o ciclista (el título promete fatalidades) y luego quien tiene más hechos de tránsito.

```
orden = cruces.sort_values(["union", "defunciones", "total"], ascending=False).copy()
# Lugar en la ZMM por la unión: lo que la página muestra cuando se filtra por
# municipio, para que el lugar dentro del municipio no se lea como el de la ZMM.
orden["lugar_zmm"] = np.arange(1, len(orden) + 1)
top = orden.head(10).copy()
top["lugar"] = range(1, 11)
top["pct_union"] = 100 * top.union / top.total
top["pct_peatones"] = 100 * top.peatones / top.total
top["pct_ciclistas"] = 100 * top.ciclistas / top.total
top["nombre"] = [nombre_cruce(d[d.id_punto_4d == k]) for k in top.index]

corte10, corte11 = int(orden.union.iloc[9]), int(orden.union.iloc[10])
```

```
print(f"Corte: el lugar 10 tiene {corte10} hechos en la unión; el 11, {corte11}. "
      + ("Sin empate en la frontera." if corte11 < corte10
         else "Hay empate en la frontera: decide el desempate."))
print(f"Los 10 suman {miles(top.union.sum())} hechos de peatón o ciclista: "
      f"{pct(100 * top.union.sum() / d.union.sum())} de la ZMM en "
      f"{100 * 10 / len(cruces):.2f} % de los cruces.\n")

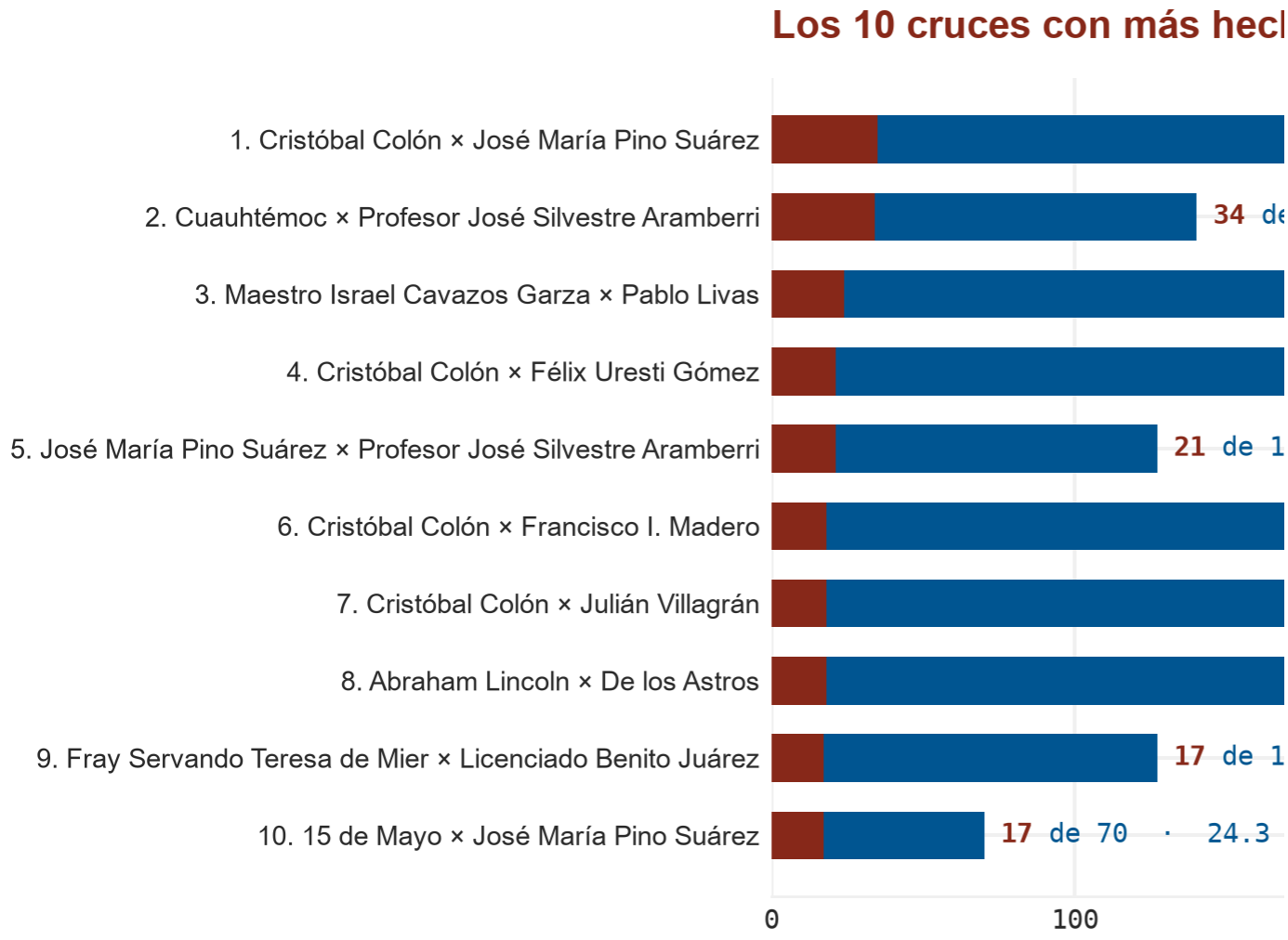
tabla = top[["nombre", "municipio", "union", "pct_union", "peatones", "ciclistas",
             "defunciones", "total", "lugar_total"]].copy()
tabla["pct_union"] = tabla.pct_union.map(pct)
tabla.columns = ["cruce", "municipio", "unión", "% unión", "peatón", "ciclista",
                 "defunciones", "hechos", "lugar por hechos"]
print(tabla.set_index(top.lugar).to_string())
```

```
Corte: el lugar 10 tiene 17 hechos en la unión; el 11, 16. Sin empate en la frontera.
Los 10 suman 223 hechos de peatón o ciclista: 2.2 % de la ZMM en 0.02 % de los cruces.

                                                      cruce  municipio  unión % unión
peatón  ciclista  defunciones  hechos  lugar por hechos
lugar
1                        Cristóbal Colón × José María Pino Suárez  Monterrey     35  11.1 %
34         1            0     314                90
2                   Cuauhtémoc × Profesor José Silvestre Aramberri  Monterrey     34  24.3 %
34         0            1     140               330
3                      Maestro Israel Cavazos Garza × Pablo Livas  Guadalupe     24   7.2 %
21         3            1     335                79
4                            Cristóbal Colón × Félix Uresti Gómez  Monterrey     21   5.6 %
20         1            1     375                60
5      José María Pino Suárez × Profesor José Silvestre Aramberri  Monterrey     21  16.5 %
21         0            0     127               385
6                           Cristóbal Colón × Francisco I. Madero  Monterrey     18   8.9 %
17         1            2     202               183
7                            Cristóbal Colón × Julián Villagrán  Monterrey     18   6.6 %
18         0            1     271               117
8                               Abraham Lincoln × De los Astros  Monterrey     18   5.4 %
17         1            0     334                81
9         Fray Servando Teresa de Mier × Licenciado Benito Juárez  Monterrey     17  13.4 %
17         0            0     127               385
10                           15 de Mayo × José María Pino Suárez  Monterrey     17  24.3 %
17         0            0      70               885
```

```
fig, ax = plt.subplots(figsize=(8.2, 4.4))
y = np.arange(10)[::-1]
ax.barh(y, top.total, color=AZUL, height=0.62, label="hechos de tránsito del cruce")
ax.barh(y, top.union, color=ROJO, height=0.62, label="con peatón o ciclista (la unión)")
for yy, (_, r) in zip(y, top.iterrows()):
    # Dos piezas, una en cada color; la segunda se corre lo que mide la primera
    # (monoespaciada a 8 pt: ~4.8 pt por carácter).
    ax.annotate(f"{r.union}", (r.total, yy), xytext=(5, 0), textcoords="offset points",
                va="center", fontsize=8, family="monospace", color=ROJO, weight="bold")
    ax.annotate(f" de {r.total}  ·  {pct(r.pct_union)}", (r.total, yy),
```

```
                xytext=(5 + 4.8 * len(str(r.union)), 0), textcoords="offset points",
                va="center", fontsize=8, family="monospace", color=AZUL)
ax.set_xlim(0, top.total.max() * 1.35)
ax.set_yticks(y, [f"{r.lugar}. {r.nombre}" for _, r in top.iterrows()], fontsize=8)
ax.set_xlabel("hechos de tránsito, 2019-2024")
ax.set_title("Los 10 cruces con más hechos de peatón o ciclista, ZMM 2019-2024")
ax.legend(loc="lower right")
pulir(ax)
for etiqueta in ax.get_yticklabels():
    etiqueta.set_fontfamily("sans-serif")
plt.show()
```



El rojo es lo que ordena; el azul es el tamaño del cruce. Por eso el orden no coincide con el lugar por hechos de tránsito: un cruce con pocos hechos pero muchos de ellos contra peatones sube; uno saturado de choques entre vehículos no.

Por municipio

La página deja elegir un municipio. El orden es el mismo, restringido a los cruces del municipio; entran solo los que tienen al menos un hecho en la unión, así que un municipio con pocos puede tener menos de diez.

```
MUNICIPIOS = sorted(d.NOM_MUN.unique())
por_mun = {m: orden[(orden.municipio == m) & (orden.union > 0)].head(10) for m in MUNICIPIOS}

resumen_mun = pd.DataFrame({
    "cruces con unión": [int(((cruces.municipio == m) & (cruces.union > 0)).sum()) for m in MUNICI
    "hechos en la unión": [int(d.union[d.NOM_MUN == m].sum()) for m in MUNICIPIOS],
    "en la página": [len(por_mun[m]) for m in MUNICIPIOS],
    "lugar 1 (unión)": [int(por_mun[m].union.iloc[0]) if len(por_mun[m]) else 0 for m in MUNICIPIO
}, index=MUNICIPIOS).sort_values("hechos en la unión", ascending=False)
print(resumen_mun.to_string())
```

	cruces con unión	hechos en la unión	en la página	lugar 1 (unión)
Monterrey	2502	4418	10	35
Guadalupe	865	1254	10	24
San Nicolás de los Garza	517	844	10	14
Apodaca	615	725	10	8
García	454	595	10	9
San Pedro Garza García	365	534	10	7
General Escobedo	427	533	10	11
Juárez	371	492	10	8
Santa Catarina	169	218	10	4
Pesquería	78	103	10	2
Cadereyta Jiménez	69	81	10	3
Salinas Victoria	41	76	10	1
General Zuazua	60	66	10	3
Santiago	1	58	1	1
Ciénega de Flores	46	54	10	2
El Carmen	32	35	10	2
Hidalgo	8	8	8	1
Abasolo	0	0	0	0

4. Una ficha por cruce

Cada ficha trae:

- **La unión**: hechos con peatón o ciclista y qué porcentaje son de los hechos de tránsito del cruce. Es la cifra que define el lugar.
- **Hechos de tránsito** totales del cruce en los seis años, y cuántos hubo en cada año.
- **Peatones** y **ciclistas**: cuántos hechos y qué porcentaje del total. Su suma puede pasar de la unión si algún hecho tuvo a los dos.
- **Hechos por trimestre**, 2019 T1 a 2024 T4.
- **La foto satelital** del cruce y su ubicación en la ZMM.
- **Lugar por hechos de tránsito** entre los cruces de la ZMM.

```
anios = list(range(2019, 2025))
trimestres = pd.period_range("2019Q1", "2024Q4", freq="Q")
d["trimestre"] = d.fecha.dt.to_period("Q")

# Todos los cruces que salen en la página: los diez de la ZMM y los diez de
# cada municipio. Se repiten mucho (los de Monterrey son casi los de la ZMM).
en_pagina = list(dict.fromkeys([*top.index, *(k for m in MUNICIPIOS for k in por_mun[m].index)]))
dp = d[d.id_punto_4d.isin(en_pagina)]
nombres = {k: nombre_cruce(s) for k, s in dp.groupby("id_punto_4d")}

series = {}
for k in en_pagina:
    s = dp[dp.id_punto_4d == k]
    series[k] = {
        "por_anio": s.groupby("ANIO").size().reindex(anios, fill_value=0).astype(int).tolist(),
        "por_anio_union": s[s.union].groupby("ANIO").size().reindex(anios, fill_value=0).astype(in
        "trim_total": s.groupby("trimestre").size().reindex(trimestres, fill_value=0).astype(int).
        "trim_union": s[s.union].groupby("trimestre").size().reindex(trimestres, fill_value=0).ast
    }
print(f"{len(en_pagina)} cruces distintos en la página")
```

```
159 cruces distintos en la página
```

La foto satelital se descarga de Esri World Imagery con `contextily` (necesita red la primera vez; cada recorte se guarda en `data/processed/satelite/` y los renders siguientes lo leen de ahí). Se recorta centrada en el cruce, 240 × 160 m, para que el aro rojo del centro caiga siempre sobre él.

```
SATELITE = cx.providers.Esri.WorldImagery
ATRIBUCION = "Imagen satelital: " + SATELITE.attribution
a_mercator = Transformer.from_crs(4326, 3857, always_xy=True)
ANCHO_M, ALTO_M = 240, 160


# Caché en disco: la página lleva ~150 cruces (~1,800 teselas a zoom 19) y Esri
# corta la conexión si se le piden todas de golpe en cada render. Es derivado
# y regenerable, así que vive en processed/.
CACHE_SAT = rutas.procesado("satelite")
CACHE_SAT.mkdir(exist_ok=True)


def satelite(k: str, lon: float, lat: float) -> Image.Image:
    """`_satelite` con caché en disco y reintentos con espera creciente."""
    archivo = CACHE_SAT / f"{k.replace(',', '_')}_{ANCHO_M}x{ALTO_M}.jpg"
    if archivo.exists():
        return Image.open(archivo).convert("RGB")
    for intento in range(6):
        try:
            foto = _satelite(lon, lat)
```

```
            break
        except requests.exceptions.RequestException:
            time.sleep(5 * 2 ** intento)
    else:
        raise RuntimeError(f"Esri no respondió para el cruce {k}")
    foto.save(archivo, quality=92)
    return foto


def _satelite(lon: float, lat: float) -> Image.Image:
    """Foto satelital centrada en el cruce, ANCHO_M x ALTO_M metros reales."""
    x, y = a_mercator.transform(lon, lat)
    # Web Mercator infla las distancias por 1/cos(lat): se corrige para que los
    # metros del recorte sean metros sobre el terreno.
    f = 1 / np.cos(np.radians(lat))
    rx, ry = ANCHO_M / 2 * f, ALTO_M / 2 * f
    img, (x0, x1, y0, y1) = cx.bounds2img(x - rx, y - ry, x + rx, y + ry,
                                          zoom=19, source=SATELITE,
                                          wait=2, max_retries=4, timeout=30)
    alto, ancho = img.shape[:2]
    col = lambda v: int(round((v - x0) / (x1 - x0) * ancho))
    fil = lambda v: int(round((y1 - v) / (y1 - y0) * alto))
    recorte = img[fil(y + ry):fil(y - ry), col(x - rx):col(x + rx), :3]
    return Image.fromarray(recorte).resize((780, 520), Image.LANCZOS)


fotos = {k: satelite(k, orden.at[k, "lon"], orden.at[k, "lat"]) for k in en_pagina}
print(f"{len(fotos)} fotos de {fotos[top.index[0]].size[0]} x {fotos[top.index[0]].size[1]} px")
```

```
159 fotos de 780 x 520 px
```

El mapa de ubicación usa las vialidades principales de OpenStreetMap que ya guarda la presentación, con los diez cruces y el de la ficha resaltado.

```
VIAS = json.loads((rutas.RAIZ / "docs/presentacion/vialidades_zmm.json")
                  .read_text(encoding="utf-8"))["vias"]


def ubicacion(ax, actual):
    for clase, coords in VIAS:
        xy = np.array(coords)
        ax.plot(xy[:, 0], xy[:, 1], color=GRAFITO, alpha=0.35 if clase == "m" else 0.2,
                lw=1.1 if clase == "m" else 0.6, solid_capstyle="round")
    ax.scatter(top.lon, top.lat, s=10, color=AZUL, zorder=3, linewidths=0)
    r = top.loc[actual]
    ax.scatter([r.lon], [r.lat], s=70, facecolor=BLANCO, edgecolor=ROJO, lw=2, zorder=4)
    ax.scatter([r.lon], [r.lat], s=12, color=ROJO, zorder=5)
    ax.set_xlim(-100.52, -100.10)
    ax.set_ylim(25.56, 25.85)
    ax.set_aspect(1 / np.cos(np.radians(25.7)))
    ax.grid(False)
```

```
    ax.set_axis_off()
    ax.set_title("Ubicación en la ZMM", fontsize=9, color=GRAFITO, pad=4)


def ficha(k):
    r, s = top.loc[k], series[k]
    fig = plt.figure(figsize=(11, 6.6))
    gs = GridSpec(3, 2, figure=fig, height_ratios=[0.9, 1.6, 1.3],
                  width_ratios=[1.55, 1], hspace=0.6, wspace=0.12,
                  left=0.04, right=0.98, top=0.84, bottom=0.12)

    fig.text(0.02, 0.975, f"{r.lugar}", fontsize=30, weight="bold", color=ROJO, va="top")
    fig.text(0.075, 0.97, r.nombre, fontsize=15, weight="bold", color=ROJO, va="top")
    fig.text(0.075, 0.925, f"{r.municipio}  ·  lugar {miles(r.lugar_total)} de "
             f"{miles(len(cruces))} cruces por hechos de tránsito",
             fontsize=9, color=GRAFITO, va="top", family="monospace")

    # Cifras
    ax = fig.add_subplot(gs[0, 0])
    ax.set_axis_off()
    tarjetas = [
        ("Peatón o ciclista (unión)", miles(r.union), f"{pct(r.pct_union)} de sus hechos", ROJO),
        ("Hechos de tránsito", miles(r.total), "2019-2024", AZUL),
        ("Con peatón", miles(r.peatones), f"{pct(r.pct_peatones)} del total", ROJO),
        ("Con ciclista", miles(r.ciclistas), f"{pct(r.pct_ciclistas)} del total", ROJO),
        ("Defunciones", miles(r.defunciones), "peatones y ciclistas", ROJO),
    ]
    for j, (titulo, cifra, sub, color) in enumerate(tarjetas):
        x = j / len(tarjetas)
        ax.text(x, 1.0, titulo.upper().replace(" (", "\n("), fontsize=6.3, color=GRAFITO, weight='
                transform=ax.transAxes, va="top")
        ax.text(x, 0.68, cifra, fontsize=18, color=color, weight="bold",
                family="monospace", transform=ax.transAxes, va="top")
        ax.text(x, 0.12, sub, fontsize=6.8, color=GRAFITO, transform=ax.transAxes,
                va="top", family="monospace")

    # Trimestres
    ax = fig.add_subplot(gs[1, 0])
    x = np.arange(len(trimestres))
    ax.bar(x, s["trim_total"], color=AZUL, width=0.75, label="hechos de tránsito")
    ax.bar(x, s["trim_union"], color=ROJO, width=0.75, label="con peatón o ciclista")
    ax.set_xticks(x[::4], [str(a) for a in anios])
    # Holgura arriba para la leyenda; son conteos, así que marcas enteras.
    ax.set_ylim(0, max(s["trim_total"]) * 1.35)
    ax.yaxis.set_major_locator(plt.MaxNLocator(integer=True, nbins=5))
    ax.set_title("Hechos de tránsito por trimestre", fontsize=10)
    ax.legend(loc="upper right", ncols=2)
    pulir(ax)

    # Por año
    ax = fig.add_subplot(gs[2, 0])
```

```
    ax.bar(anios, s["por_anio"], color=AZUL, width=0.6)
    ax.bar(anios, s["por_anio_union"], color=ROJO, width=0.6)
    debajo = ax.get_xaxis_transform()
    for a, v, u in zip(anios, s["por_anio"], s["por_anio_union"]):
        ax.text(a, v, miles(v), ha="center", va="bottom", fontsize=8,
                family="monospace", color=AZUL, weight="bold")
        # El total de la unión va debajo del año, en rojo.
        ax.text(a, -0.32, miles(u), ha="center", va="top", fontsize=8,
                family="monospace", color=ROJO, weight="bold", transform=debajo)
    ax.set_xticks(anios)
    ax.set_yticks([])
    ax.set_title("Hechos de tránsito por año · en rojo, con peatón o ciclista", fontsize=10)
    ax.grid(False)
    pulir(ax)

    # Foto satelital
    ax = fig.add_subplot(gs[1:, 1])
    ax.imshow(fotos[k], extent=(-ANCHO_M / 2, ANCHO_M / 2, -ALTO_M / 2, ALTO_M / 2))
    ax.scatter([0], [0], s=420, facecolor="none", edgecolor=BLANCO, lw=3.2)
    ax.scatter([0], [0], s=420, facecolor="none", edgecolor=ROJO, lw=1.8)
    ax.set_axis_off()
    ax.set_title("Vista satelital, 240 × 160 m", fontsize=9, color=GRAFITO, pad=4)

    # Ubicación, arriba de la foto
    ubicacion(fig.add_subplot(gs[0, 1]), k)

    fig.text(0.02, 0.01, ATRIBUCION, fontsize=6, color=GRAFITO, alpha=0.7)
    # JPEG y no PNG: la foto satelital en PNG hace que el PDF pese 20 MB.
    buf = io.BytesIO()
    fig.savefig(buf, format="jpg", dpi=150, pil_kwargs={"quality": 85})
    plt.close(fig)
    display(ImagenIPy(buf.getvalue()))
```

```
for k in top.index:
    ficha(k)
```

1 Cristóbal Colón × José María Pino Suárez

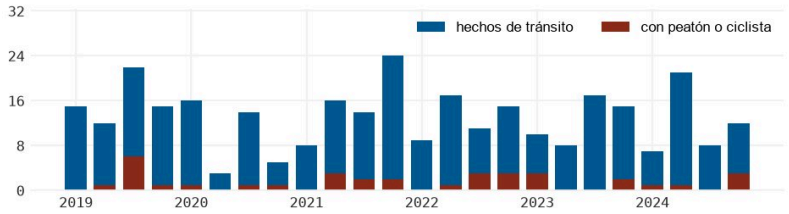
Monterrey · lugar 90 de 51,305 cruces por hechos de tránsito

PEATÓN O CICLISTA (UNIÓN)	HECHOS DE TRÁNSITO	CON PEATÓN	CON CICLISTA	DEFUNCIONES
35	314	34	1	0
11.1 % de sus hechos	2019-2024	10.8 % del total	0.3 % del total	peatones y ciclistas

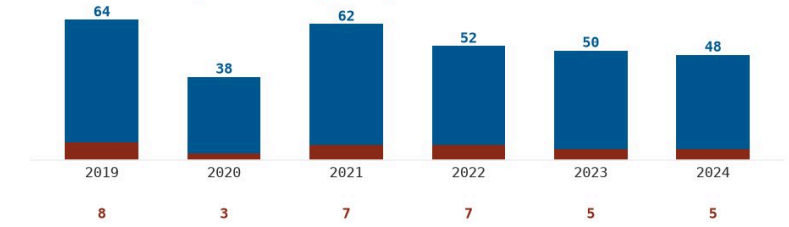
Ubicación en la ZMM



Hechos de tránsito por trimestre



Hechos de tránsito por año · en rojo, con peatón o ciclista



Vista satelital, 240 × 160 m



Imagen satelital: Tiles (C) Esri -- Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User Community

2 Cuauhtémoc × Profesor José Silvestre Aramberri

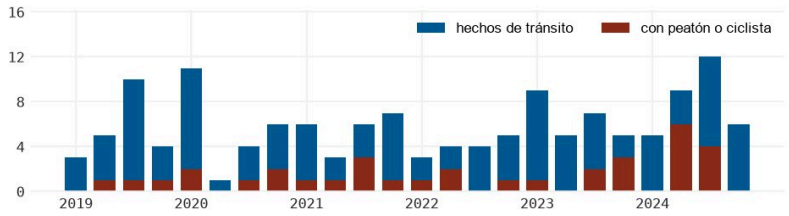
Monterrey · lugar 330 de 51,305 cruces por hechos de tránsito

PEATÓN O CICLISTA (UNIÓN)	HECHOS DE TRÁNSITO	CON PEATÓN	CON CICLISTA	DEFUNCIONES
34	140	34	0	1
24.3 % de sus hechos	2019-2024	24.3 % del total	0.0 % del total	peatones y ciclistas

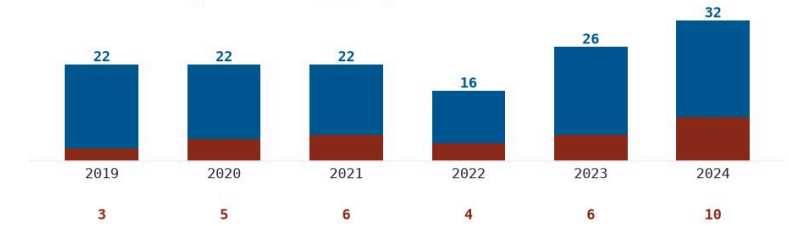
Ubicación en la ZMM



Hechos de tránsito por trimestre



Hechos de tránsito por año · en rojo, con peatón o ciclista



Vista satelital, 240 × 160 m



Imagen satelital: Tiles (C) Esri -- Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User Community

3 Maestro Israel Cavazos Garza × Pablo Livas

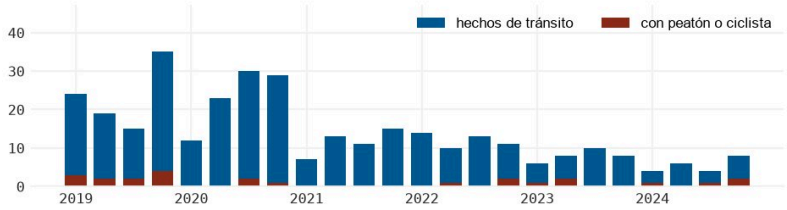
Guadalupe · lugar 79 de 51,305 cruces por hechos de tránsito

PEATÓN O CICLISTA (UNIÓN)	HECHOS DE TRÁNSITO	CON PEATÓN	CON CICLISTA	DEFUNCIONES
24	335	21	3	1
7.2 % de sus hechos	2019-2024	6.3 % del total	0.9 % del total	peatones y ciclistas

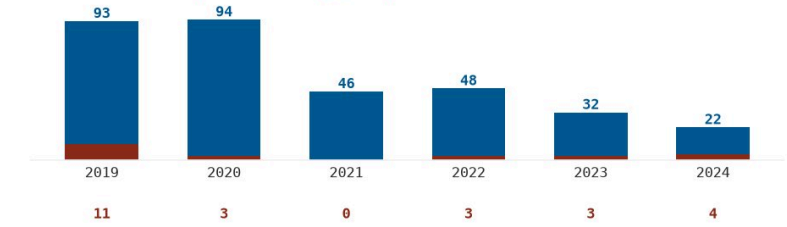
Ubicación en la ZMM



Hechos de tránsito por trimestre



Hechos de tránsito por año · en rojo, con peatón o ciclista



Vista satelital, 240 × 160 m



Imagen satelital: Tiles (C) Esri -- Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User Community

4 Cristóbal Colón × Félix Uresti Gómez

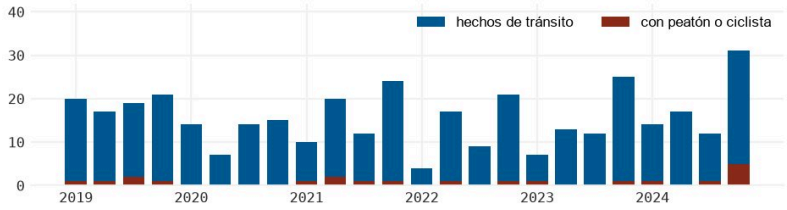
Monterrey · lugar 60 de 51,305 cruces por hechos de tránsito

PEATÓN O CICLISTA (UNIÓN)	HECHOS DE TRÁNSITO	CON PEATÓN	CON CICLISTA	DEFUNCIONES
21	375	20	1	1
5.6 % de sus hechos	2019-2024	5.3 % del total	0.3 % del total	peatones y ciclistas

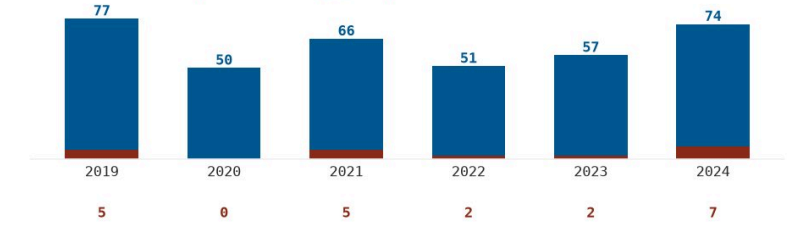
Ubicación en la ZMM



Hechos de tránsito por trimestre



Hechos de tránsito por año · en rojo, con peatón o ciclista



Vista satelital, 240 × 160 m

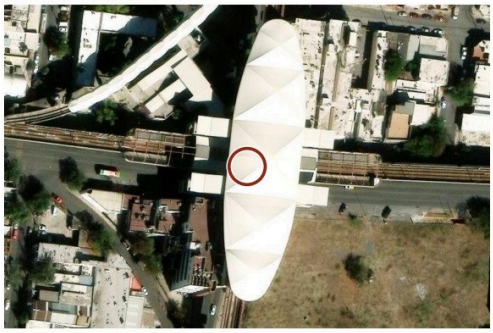


Imagen satelital: Tiles (C) Esri -- Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User Community

5 José María Pino Suárez × Profesor José Silvestre Aramberri

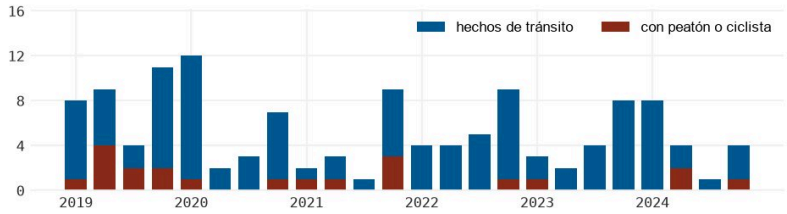
Monterrey · lugar 385 de 51,305 cruces por hechos de tránsito

PEATÓN O CICLISTA (UNIÓN)	HECHOS DE TRÁNSITO	CON PEATÓN	CON CICLISTA	DEFUNCIONES
21	127	21	0	0
16.5 % de sus hechos	2019-2024	16.5 % del total	0.0 % del total	peatones y ciclistas

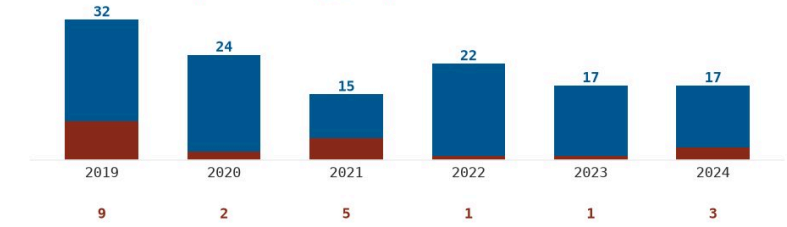
Ubicación en la ZMM



Hechos de tránsito por trimestre



Hechos de tránsito por año · en rojo, con peatón o ciclista



Vista satelital, 240 × 160 m



Imagen satelital: Tiles (C) Esri -- Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User Community

6 Cristóbal Colón × Francisco I. Madero

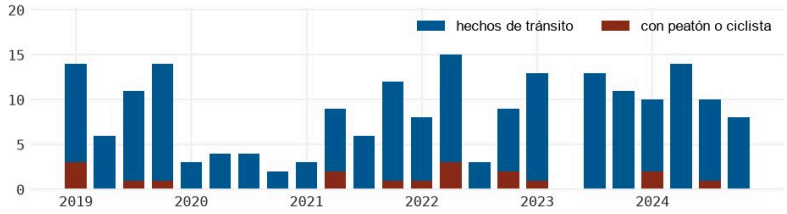
Monterrey · lugar 183 de 51,305 cruces por hechos de tránsito

PEATÓN O CICLISTA (UNIÓN)	HECHOS DE TRÁNSITO	CON PEATÓN	CON CICLISTA	DEFUNCIONES
18	202	17	1	2
8.9 % de sus hechos	2019-2024	8.4 % del total	0.5 % del total	peatones y ciclistas

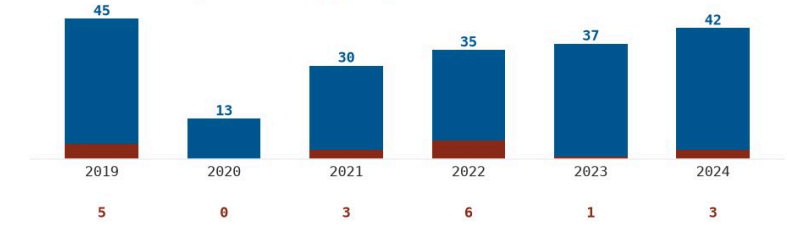
Ubicación en la ZMM



Hechos de tránsito por trimestre



Hechos de tránsito por año · en rojo, con peatón o ciclista



Vista satelital, 240 × 160 m



Imagen satelital: Tiles (C) Esri -- Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User Community

7 Cristóbal Colón × Julián Villagrán

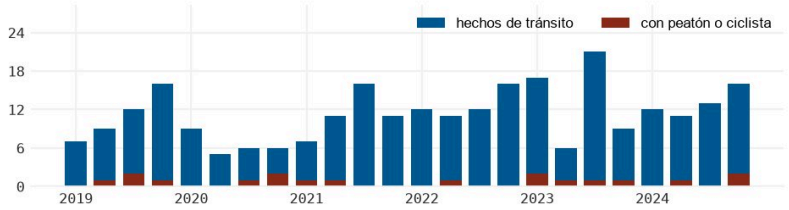
Monterrey · lugar 117 de 51,305 cruces por hechos de tránsito

PEATÓN O CICLISTA (UNIÓN)	HECHOS DE TRÁNSITO	CON PEATÓN	CON CICLISTA	DEFUNCIONES
18	271	18	0	1
6.6 % de sus hechos	2019-2024	6.6 % del total	0.0 % del total	peatones y ciclistas

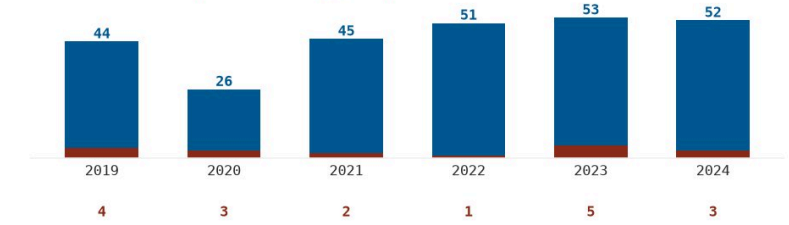
Ubicación en la ZMM



Hechos de tránsito por trimestre



Hechos de tránsito por año · en rojo, con peatón o ciclista



Vista satelital, 240 × 160 m

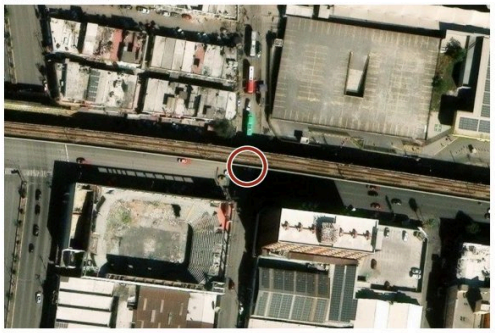


Imagen satelital: Tiles (C) Esri -- Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User Community

8 Abraham Lincoln × De los Astros

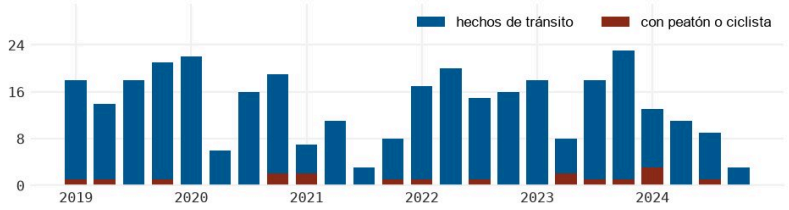
Monterrey · lugar 81 de 51,305 cruces por hechos de tránsito

PEATÓN O CICLISTA (UNIÓN)	HECHOS DE TRÁNSITO	CON PEATÓN	CON CICLISTA	DEFUNCIONES
18	334	17	1	0
5.4 % de sus hechos	2019-2024	5.1 % del total	0.3 % del total	peatones y ciclistas

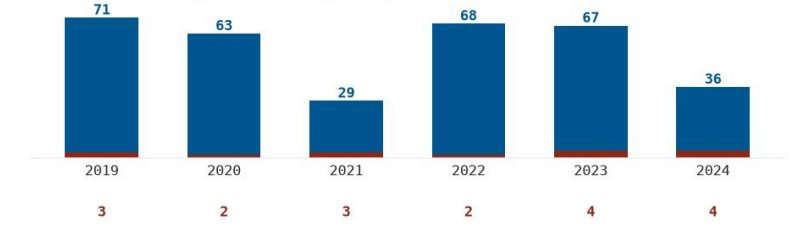
Ubicación en la ZMM



Hechos de tránsito por trimestre



Hechos de tránsito por año · en rojo, con peatón o ciclista



Vista satelital, 240 × 160 m

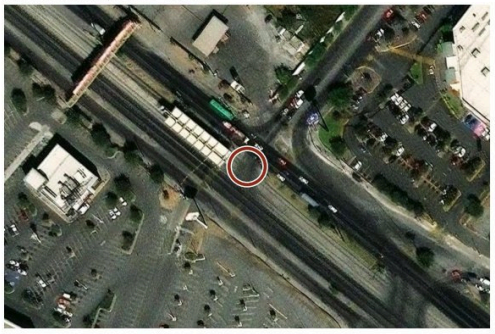


Imagen satelital: Tiles (C) Esri -- Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User Community

9 Fray Servando Teresa de Mier × Licenciado Benito Juárez

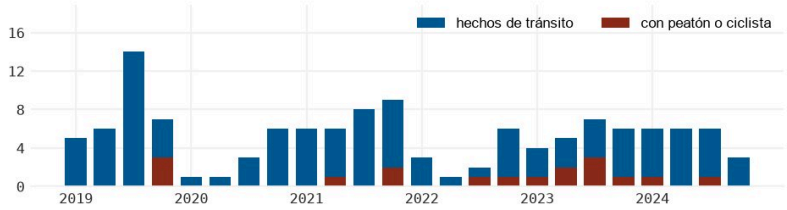
Monterrey · lugar 385 de 51,305 cruces por hechos de tránsito

PEATÓN O CICLISTA (UNIÓN)	HECHOS DE TRÁNSITO	CON PEATÓN	CON CICLISTA	DEFUNCIONES
17	127	17	0	0
13.4 % de sus hechos	2019-2024	13.4 % del total	0.0 % del total	peatones y ciclistas

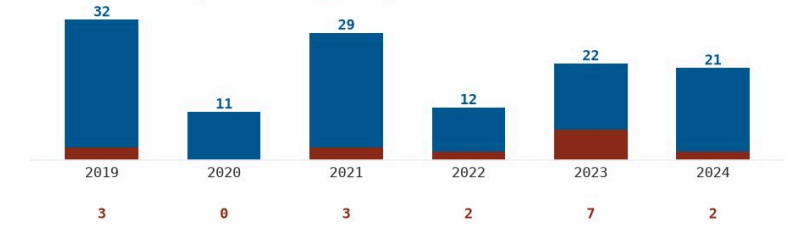
Ubicación en la ZMM



Hechos de tránsito por trimestre



Hechos de tránsito por año · en rojo, con peatón o ciclista



Vista satelital, 240 × 160 m

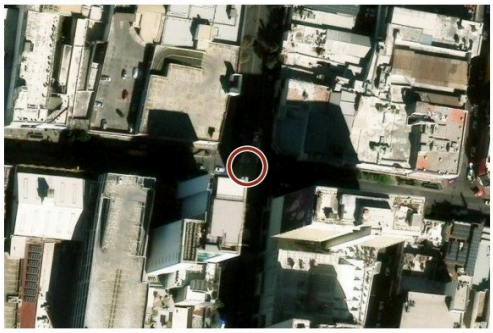


Imagen satelital: Tiles (C) Esri -- Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User Community

10 15 de Mayo × José María Pino Suárez

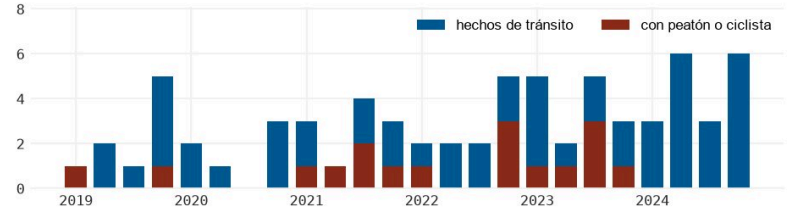
Monterrey · lugar 885 de 51,305 cruces por hechos de tránsito

PEATÓN O CICLISTA (UNIÓN)	HECHOS DE TRÁNSITO	CON PEATÓN	CON CICLISTA	DEFUNCIONES
17	70	17	0	0
24.3 % de sus hechos	2019-2024	24.3 % del total	0.0 % del total	peatones y ciclistas

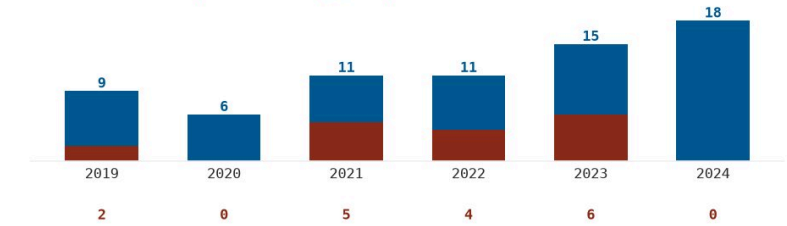
Ubicación en la ZMM



Hechos de tránsito por trimestre



Hechos de tránsito por año · en rojo, con peatón o ciclista



Vista satelital, 240 × 160 m



Imagen satelital: Tiles (C) Esri -- Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User Community

5. La página HTML

Las mismas fichas, como página suelta con el estilo de la presentación: `docs/cruces_peatones/index.html`. Se arma igual que la presentación: el notebook deja los datos en un JSON, y una plantilla los dibuja en el navegador. Las fotos y las tipografías van incrustadas, así que se abre con doble clic y funciona sin conexión (salvo la liga a Google Maps).

La página trae una **vista por municipio** además de la de toda la ZMM. Cada vista lleva su resumen —cruces, hechos de tránsito y fallecidos, en total y con peatón o ciclista— y la lista de sus cruces. Los cruces se guardan una sola vez y las vistas los citan por posición, porque se repiten entre vistas.

```
DESTINO = rutas.RAIZ / "docs" / "cruces_peatones"
FUENTES = rutas.RAIZ / "docs" / "presentacion" / "fuentes"


def jpeg_uri(imagen: Image.Image) -> str:
    # Más chica que en el notebook: la página lleva una foto por cruce de
    # todas las vistas, y a 600 px de ancho sigue viéndose nítida en la ficha.
    buf = io.BytesIO()
    imagen.resize((600, 400), Image.LANCZOS).save(
        buf, format="JPEG", quality=72, optimize=True, progressive=True)
    return "" + base64.b64encode(buf.getvalue()).decode("ascii")


def resumen(hechos: pd.DataFrame, nodos_vista: pd.DataFrame) -> dict:
    """Las seis cifras del encabezado: tres grupos, total y peatón o ciclista."""
    return {
        "cruces": int(len(nodos_vista)),
        "crucesUnion": int((nodos_vista.union > 0).sum()),
        "hechos": int(len(hechos)),
        "union": int(hechos.union.sum()),
        # Fallecidos: personas, todas las calidades (conductor, pasajero, peatón,
        # ciclista, otro) contra solo peatones y ciclistas.
        "fallecidos": int(hechos.TOTMUERTOS.sum()),
        "fallecidosPC": int(hechos.defunciones_pc.sum()),
    }


indice = {k: j for j, k in enumerate(en_pagina)}
vistas = [{"nombre": "Todos", **resumen(d, cruces), "ids": [indice[k] for k in top.index]}]
for m in MUNICIPIOS:
    vistas.append({"nombre": m, **resumen(d[d.NOM_MUN == m], cruces[cruces.municipio == m]),
                   "ids": [indice[k] for k in por_mun[m].index]})

datos = {
    "corte": [corte10, corte11],
    "anios": anios,
    "trimestres": [f"{p.year} T{p.quarter}" for p in trimestres],
    "vias": VIAS,
    "atribucion": ATRIBUCION,
    "ventana": [ANCHO_M, ALTO_M],
    "vistas": vistas,
```

```
    "cruces": [{
        "nombre": nombres[k], "municipio": r.municipio,
        "lon": round(float(r.lon), 5), "lat": round(float(r.lat), 5),
        "total": int(r.total), "union": int(r.union),
        "peatones": int(r.peatones), "ciclistas": int(r.ciclistas),
        "defunciones": int(r.defunciones),
        "lugarTotal": int(r.lugar_total), "lugarZMM": int(r.lugar_zmm),
        "porAnio": series[k]["por_anio"], "porAnioUnion": series[k]["por_anio_union"],
        "trimTotal": series[k]["trim_total"], "trimUnion": series[k]["trim_union"],
        "foto": jpeg_uri(fotos[k]),
    } for k, r in orden.loc[en_pagina].iterrows()],
}
print(f"Fallecidos ZMM: {miles(vistas[0]['fallecidos'])} | peatón o ciclista: "
      f"{miles(vistas[0]['fallecidosPC'])}")

plantilla = (DESTINO / "plantilla.html").read_text(encoding="utf-8")
html = plantilla.replace("{{DATOS}}", json.dumps(datos, ensure_ascii=False, separators=(",", ":")))
html = re.sub(
    r"\{\{FUENTE:([^}]+)\}\}",
    lambda m: "data:font/woff2; base64,"
              + base64.b64encode((FUENTES / f"{m.group(1)}.woff2").read_bytes()).decode("ascii"),
    html,
)
(DESTINO / "index.html").write_text(html, encoding="utf-8")
print(f"docs/cruces_peatones/index.html: {len(html.encode('utf-8')) / 1e6:.2f} MB")
```

```
Fallecidos ZMM: 1,167 | peatón o ciclista: 475
docs/cruces_peatones/index.html: 11.28 MB
```

Limitaciones

Limitación	Qué implica
Conteos, no tasas	No hay aforo peatonal ni vehicular por cruce. Un cruce del centro con mucha gente a pie tendrá más atropellamientos aunque no sea más peligroso por persona que cruza.
Las defunciones son pocas por cruce	El máximo de la ZMM son tres defunciones de peatón o ciclista en un cruce en seis años (§ 2). El ranking se ordena por la unión y las defunciones solo desempatan: el título habla de fatalidades, pero el orden lo dan sobre todo los atropellamientos.

Limitación	Qué implica
El nodo de 10 m	Un cruce muy ancho (glorieta, cruce con lateral) puede repartirse en más de un nodo. La celda de abajo cuenta, para cada uno de los diez, los hechos de la unión que caen en otros nodos a menos de 40 m.

```
# Comprobación de la limitación anterior: lo que se quedaría fuera de cada
# cruce si el nodo de 10 m lo partió.
cerca = []
for k, r in top.iterrows():
    dist = np.hypot((d.LONGITUD - r.lon) * 111_320 * np.cos(np.radians(r.lat)),
                    (d.LATITUD - r.lat) * 110_570)
    vecinos = d[(dist < 40) & (d.id_punto_4d != k)]
    cerca.append((r.lugar, r.nombre, int(r.union), int(vecinos.union.sum())))
print(pd.DataFrame(cerca, columns=["lugar", "cruce", "unión", "unión en vecinos < 40 m"])
      .set_index("lugar").to_string())
```

```
                                                cruce  unión  unión en vecinos < 40 m
lugar
1                          Cristóbal Colón × José María Pino Suárez     35                        1
2                  Cuauhtémoc × Profesor José Silvestre Aramberri     34                        0
3                      Maestro Israel Cavazos Garza × Pablo Livas     24                        0
4                           Cristóbal Colón × Félix Uresti Gómez     21                        0
5       José María Pino Suárez × Profesor José Silvestre Aramberri     21                        0
6                            Cristóbal Colón × Francisco I. Madero     18                        0
7                               Cristóbal Colón × Julián Villagrán     18                        0
8                                  Abraham Lincoln × De los Astros     18                        2
9          Fray Servando Teresa de Mier × Licenciado Benito Juárez     17                        0
10                            15 de Mayo × José María Pino Suárez     17                        0
```

La foto no es del periodo | La imagen satelital es la vigente en Esri, no la de 2019-2024: puede mostrar obras o cambios posteriores. |